HR Auto

Database on AWS (Amazon RDS for MySQL)

Setup, migration and operating procedure

Prepared by: Ananya Pandey

Date: 27 October 2026

Applies to: HR Auto (Resume / CV Extractor, candidate portal and hiring dashboard), branch `mysql-migration`

Status: Procedure rehearsed end to end on MySQL 8.4.7 on 27 October 2026 (see section 9).

1. Summary 2
2. Current setup 2
3. What is in the database 3
4. AWS setup 3
5. Migration procedure 4
6. Application configuration 6
7. Rollback plan 7
8. Backups, monitoring and file storage 7
9. Rehearsal results (27 October 2026) 7
10. Decisions needed 8

1. Summary

HR Auto's database runs today on a MySQL 8.4 server on a development laptop. This document describes how to move it to **Amazon RDS for MySQL** so the application can run in production: what the database contains, how to set up RDS, the exact commands to copy the data across, how to check the copy, and how to run and back it up afterwards.

- **What moves:** one MySQL database, `hr_auto` (7 tables and 1 view). It is small today and grows with every uploaded resume and job application.
- **What does not move:** resume files. They are stored in Google Cloud Storage; the database only stores each file's location (key). See section 8.
- **Schema is automatic:** the application creates and upgrades its own tables from versioned migration files. No one needs to write or run table-creation SQL by hand.
- **Downtime:** a few minutes. Uploads and applications are paused only while the data is exported and loaded.
- **Risk:** low. The local database is left untouched, so going back is a single setting change (section 7).

2. Current setup

Item	Today
Database engine	MySQL Community Server 8.4.7, database `hr_auto`, utf8mb4 / `utf8mb4_0900_ai_ci`, UTC
Where it runs	Local laptop (portable MySQL install), port 3306
Application (API)	Node.js 20+ / Express / TypeScript (`server/`)
Web app	React + Vite (`frontend/`): admin dashboard, careers page and application form
Schema management	SQL migration files `server/src/db/migrations/001`–`005`, applied automatically on start-up and recorded in `schema_migrations`
Connection settings	Environment variables `DATABASE_URL`, `DATABASE_SSL`, `DATABASE_SSL_CA`, `DATABASE_POOL_MAX`, `DATABASE_AUTO_MIGRATE`
File storage	Google Cloud Storage bucket (uploaded resumes and Excel exports)
Sign-in	One admin account from environment variables; no users are stored in the database

3. What is in the database

Row counts are from the local database on 27 October 2026.

Table	What it holds	Rows
`records`	One row per uploaded resume: file details, department, status, all 56 extracted fields (JSON), full resume text, processing log. Also acts as the work queue for resume extraction.	1

Table	What it holds	Rows
`applications`	One row per job application submitted through the careers page: personal, education and professional details, applied role, resume file details, hiring status.	3
`application_notes`	Internal hiring notes on an application. Never shown to the candidate.	1
`application_emails`	Every email sent to a candidate, with its delivery status.	0
`audit_log`	Append-only trail of admin actions (status changes, emails, departments added).	3
`departments`	Departments a resume can be filed under. Seeded with 7; the admin can add more from the upload screen.	8
`schema_migrations`	Which migration files have been applied. Managed by the application.	5
`records_candidates` (view)	Read-only, one-row-per-resume view for people browsing the data with SQL tools. The app does not use it.	—

> **Personal data:** `records` and `applications` contain candidate PII (names, emails, phone numbers, full resume text). Treat the database, its backups and every export file as confidential.

4. AWS setup

4.1 Engine and instance

Setting	Recommendation
Engine	**Amazon RDS for MySQL 8.4** (same major version as today; 8.0.19 or newer is the minimum the schema needs)
Region	The same region as the API server, to keep query round trips short
Instance class	`db.t4g.small` to start (the data set is small); scale up later without data changes
Storage	gp3, 20 GB, storage autoscaling on
Availability	Single-AZ for testing; **Multi-AZ** for production
Encryption	Encryption at rest **on** (KMS); TLS required for every connection
Backups	Automated backups **on**, retention 7 days or more
Public access	**No.** Place it in private subnets (section 4.3)

4.2 Parameter group

Create a custom DB parameter group (family `mysql8.4`) with these values and attach it to the instance. The application relies on them.

Parameter	Value	Why
`character_set_server`	`utf8mb4`	Names and resumes contain non-English characters and emoji

Parameter	Value	Why
`collation_server`	`utf8mb4_0900_ai_ci`	Matches the schema; case-insensitive search
`time_zone`	UTC (RDS default)	All timestamps are stored in UTC
`transaction_isolation`	`READ-COMMITTED`	Avoids lock contention on the resume-processing queue (the app also sets it per connection)
`group_concat_max_len`	`1048576`	Otherwise the `records_candidates` view cuts skill lists at 1 KB
`max_allowed_packet`	`67108864` (64 MB)	A large resume row is sent as one packet
`require_secure_transport`	`ON`	Refuse unencrypted connections
`sql_mode`	Default	Must not include `NO_BACKSLASH_ESCAPES` (the data export uses backslash escapes)

4.3 Network and access

- Put the RDS instance in **private subnets** of a VPC. Its security group allows port 3306 **only** from the API's security group (and a bastion / SSM host for administration).
- **Recommended:** run the API on AWS too (ECS Fargate or App Runner) in the same VPC. The API is a long-running process with a background worker, so AWS Lambda does not suit it.
- If the API stays on Render for now, RDS needs a public endpoint restricted to Render's static outbound IPs (a paid Render feature), with TLS mandatory. This is less isolated and best treated as temporary.
- For administration from a laptop, use **AWS Systems Manager port forwarding** or an EC2 bastion host rather than opening the database to the internet.

4.4 Credentials

- Store the database password, `ADMIN_PASSWORD`, `SESSION_SECRET` and the Bedrock key in **AWS Secrets Manager** and inject them into the API as environment variables.
- The application connects as a dedicated user, `hr_auto`, not as the RDS master user (section 5, step 2).
- Download the RDS certificate bundle `global-bundle.pem` from AWS for command-line connections. The API already contains this bundle and verifies the server certificate when `DATABASE_SSL=1`.

5. Migration procedure

Commands below are for Windows PowerShell on the machine that holds the local database, and for any machine that can reach RDS (bastion or SSM tunnel). Replace values in angle brackets.

Step 1 — Create the RDS instance

1. Create the parameter group from section 4.2.
2. Create the RDS for MySQL 8.4 instance with the settings in section 4.1, attached to that parameter group and the private security group.
3. Wait until the status is **Available** and note the endpoint, e.g. `hr-auto.xxxxxxxx.ap-south-1.rds.amazonaws.com`.

Step 2 — Create the database and application user

Connect as the RDS master user and run:

```
CREATE DATABASE hr_auto
  CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;

CREATE USER 'hr_auto'@'%' IDENTIFIED BY '<strong password>' REQUIRE SSL;

GRANT SELECT, INSERT, UPDATE, DELETE,
      CREATE, ALTER, INDEX, DROP, REFERENCES, CREATE VIEW, SHOW VIEW
  ON hr_auto.* TO 'hr_auto'@'%';
```

The schema-change privileges (CREATE, ALTER, INDEX, DROP, REFERENCES, CREATE VIEW) are needed because the application applies its own migrations. They are limited to the `hr_auto` database.

Step 3 — Create the tables (empty schema)

From the project's `server` folder, run the migrations against RDS:

```
$env:DATABASE_URL = "mysql://hr_auto:<password>@<rds-endpoint>:3306/hr_auto"
$env:DATABASE_SSL = "1"
npm run migrate
```

Expected output: migrations `001_mysql_baseline.sql` to `005_drop_department_check.sql` applied, then `database schema up to date`. The 7 default departments are created by migration 004.

Step 4 — Export the data from the local database

1. Stop the local API (and anything else writing to the database) so no new uploads arrive during the copy.
2. Export **data only** (the schema already exists on RDS):

```
mysqldump -u root -h 127.0.0.1 `
  --no-create-info --skip-triggers --single-transaction `
  --insert-ignore --hex-blob --default-character-set=utf8mb4 `
  --set-gtid-purged=OFF `
  --ignore-table=hr_auto.schema_migrations `
  --result-file=hr_auto_data.sql hr_auto
```

> **Note:** use `--result-file`, not `> file.sql`. In Windows PowerShell 5.1, `>` writes UTF-16 and the file will not load. `--insert-ignore` lets the 7 default departments (already on RDS) be skipped instead of failing.

Step 5 — Load the data into RDS

1. Copy `hr_auto_data.sql` to the machine that can reach RDS (encrypted S3 bucket or `scp` to the bastion).
2. Load it:

```
mysql -h <rds-endpoint> -u hr_auto -p `
  --ssl-ca=global-bundle.pem --ssl-mode=VERIFY_IDENTITY `
  --default-character-set=utf8mb4 hr_auto `
  -e "source hr_auto_data.sql"
```

Step 6 — Verify the copy

Run this on **both** the local database and RDS. Every line must match.

```
SELECT 'records', COUNT(*) FROM records
UNION ALL SELECT 'applications', COUNT(*) FROM applications
UNION ALL SELECT 'application_notes', COUNT(*) FROM application_notes
UNION ALL SELECT 'application_emails', COUNT(*) FROM application_emails
UNION ALL SELECT 'audit_log', COUNT(*) FROM audit_log
UNION ALL SELECT 'departments', COUNT(*) FROM departments
UNION ALL SELECT 'records checksum',
  SUM(CRC32(CONCAT_WS('|', id, checksum, fields, text_content))) FROM records
UNION ALL SELECT 'applications checksum',
  SUM(CRC32(CONCAT_WS('|', id, email, skills))) FROM applications;
```

Step 7 — Switch the application to RDS

1. Set the API's environment variables (section 6) to point at RDS and restart it.
2. Check `GET /api/health/ready` returns `{"status":"ready","database":"ok"}`.
3. Sign in to the dashboard, open a few resumes and applications, and submit one test application on the careers page.

Step 8 — Finish

1. Take a **manual RDS snapshot** named e.g. `hr-auto-after-migration`.
2. **Delete the export file** `hr_auto_data.sql` from every machine and bucket it was copied to. It contains candidate PII.
3. Keep the local database untouched for at least one week as a fallback.

> **Starting fresh instead:** if the local data is only test data, skip steps 4–6. Step 3 already gives RDS a complete, empty, ready-to-use schema.

6. Application configuration

Environment variable	Value on AWS
`DATABASE_URL`	`mysql://hr_auto:<password>@<rds-endpoint>:3306/hr_auto` (password from Secrets Manager)
`DATABASE_SSL`	`1` (verified TLS using the bundled RDS certificate authority)
`DATABASE_SSL_CA`	Leave empty (only needed for a non-RDS certificate)
`DATABASE_POOL_MAX`	`10`. Keep (number of API instances × 10) well below the instance's `max_connections`
`DATABASE_AUTO_MIGRATE`	`1` for a single API instance. With several instances, set `0` and run `npm run migrate` once per deploy
Everything else	Unchanged (admin login, session, AI extraction, email and storage settings do not depend on the database)

Future schema changes need no manual database work: a developer adds a new numbered SQL file to `server/src/db/migrations/`, and it is applied once, automatically, on the next deploy.

7. Rollback plan

- Before cut-over: nothing to undo. RDS can be deleted and recreated.

- After cut-over: point `DATABASE_URL` back at the previous database and restart the API. Any data written to RDS after the switch would need to be copied back with the same export/load commands (steps 4–6 in reverse).
- Point-in-time restore from RDS automated backups covers mistakes made later (e.g. accidental deletion).

8. Backups, monitoring and file storage

8.1 Backups

- Automated backups with 7+ days retention (point-in-time restore).
- Manual snapshot after the migration and before any major release.
- Backups and snapshots are encrypted because the instance is encrypted.

8.2 Monitoring (CloudWatch alarms)

Metric	Suggested alarm
`FreeStorageSpace`	Below 20% of allocated storage
`CPUUtilization`	Above 80% for 15 minutes
`DatabaseConnections`	Above 80% of `max_connections`
API `/api/health/ready`	Use as the load balancer / App Runner health check; it fails when the database is unreachable

8.3 Resume files

The database stores only each file's key (for example `resumes/linkedin/a1b2c3d4e5f6.pdf`); the files themselves stay in the Google Cloud Storage bucket. The database move does **not** require moving files. If files should also move to **Amazon S3**, copy the bucket with the same keys and add an S3 storage driver to the API (`server/src/storage/`); this is a separate piece of work.

9. Rehearsal results (27 October 2026)

Steps 3 to 6 were rehearsed on the local MySQL 8.4.7 server, loading into a new empty database:

- All 5 migrations applied cleanly to an empty database.
- The data-only export ran without errors; the 7 default departments already created by migration 004 were skipped as intended, and the custom department was copied.
- Row counts for every table **and** the content checksums for `records` and `applications` matched exactly between source and copy.
- The `records_candidates` view returned the same rows on the copy.
- The rehearsal database and export file were deleted afterwards.

10. Decisions needed

Decision	Options / recommendation
Where the API runs	**AWS ECS Fargate or App Runner** in the RDS VPC (recommended), or stay on Render with a public, IP-restricted RDS endpoint

Decision	Options / recommendation
AWS region	Same region as the API (e.g. `ap-south-1` Mumbai if users are in India)
High availability	Single-AZ (lower cost) or Multi-AZ (automatic failover) for production
Resume files	Keep on Google Cloud Storage, or move to S3 (separate task)
Existing local data	Migrate it (steps 4–6) or start production empty
Who holds credentials	Owner of the AWS account / Secrets Manager entries